FASE 3 · PROYECTO 07 · RUST

rhttpd

Un servidor HTTP/1.1 en Rust:
seguridad de memoria sin renunciar al control

Versión 1.0 · Roadmap Sistemas Linux

Resumen

Vas a escribir un servidor HTTP/1.1 de ficheros estáticos en Rust usando solo la biblioteca estándar: tu propio pool de hilos, un parser HTTP robusto frente a entradas maliciosas, conexiones persistentes, timeouts y apagado ordenado. El objetivo es aprender a pensar como el compilador de Rust: quién es el dueño de cada dato, cuánto vive y quién puede tocarlo a la vez, y comprobar que los bugs que perseguías con Valgrind en C aquí no compilan.

CAPÍTULO I

Introducción

Rust está entrando en lugares antes reservados a C: el kernel de Linux, Android, Windows, herramientas de infraestructura… Su propuesta es simple de enunciar y difícil de interiorizar: el compilador verifica en tiempo de compilación que no hay *use-after-free*, *double free*, punteros colgantes ni *data races* en el código seguro. El precio es aprender el sistema de *ownership* y *borrowing*.

Has elegido un servidor HTTP porque toca todo lo difícil a la vez: E/S de red con lecturas parciales (como en `rawio`), concurrencia (como en `ringdev`), parsing de entradas hostiles y gestión de recursos con vida limitada.

CAPÍTULO II

Instrucciones generales

- Rust estable, edición 2021 o posterior. Proyecto creado con `cargo`.
- Para la parte obligatoria solo se permite la biblioteca estándar, más el crate `signal-hook` para capturar Ctrl+C. **Prohibidos** `tokio`, `hyper`, `rayon`, `crossbeam` y similares.
- `#![forbid(unsafe_code)]` en la raíz del crate. Si crees que necesitas `unsafe`, estás resolviendo mal el problema.
- `cargo fmt --check`, `cargo clippy -- -D warnings` y `cargo test` en verde. Configura GitHub Actions para ello.

- Ningún `unwrap()` ni `expect()` en caminos que dependan de la red o del disco: una petición malformada nunca puede tumbar el servidor (`panic` = 0 en la evaluación).

CAPÍTULO III

Parte obligatoria

1. Uso

```
$ rhttpd --root ./www --port 8080 --threads 8
rhttpd: serving ./www on 0.0.0.0:8080 with 8 workers
```

2. Pool de hilos

- Implementa tu propio pool con un número fijo de hilos y una cola de trabajos compartida (`std::sync::mpsc` con `Arc<Mutex<Receiver<...>>>`, o `Mutex` + `Condvar`).
- Cuando la cola supera un límite configurable, las conexiones nuevas reciben un `503 Service Unavailable` en lugar de acumularse sin control (*backpressure*).
- En el README, explica por qué el compilador te obliga a envolver el receptor en `Arc<Mutex<…>>` y qué garantizan los *traits* `Send` y `Sync`.

3. Parser HTTP

El parser es un módulo independiente, sin E/S, que recibe bytes y devuelve una petición o un error. Esto lo hace testeable y *fuzzeable*.

- Soporta peticiones que llegan **fragmentadas** en varias lecturas de TCP y varias peticiones en una misma lectura (*pipelining*). Diseña el parser como una máquina de estados sobre un buffer.
- Valida estrictamente: línea de petición, versión (`HTTP/1.0` y `HTTP/1.1`), cabeceras `Nombre: valor`, `Host` obligatorio en 1.1.
- Límites: línea de petición ≤ 8 KiB, cabeceras ≤ 64 en total y ≤ 16 KiB. Si se exceden: `414` o `431`.
- Decodifica el *percent-encoding* de la ruta (`%20`) y rechaza bytes nulos.

4. Servir ficheros

- Métodos `GET` y `HEAD`. Cualquier otro: `405` con la cabecera `Allow`.
- Para un directorio, sirve su `index.html` si existe; si no, `403`.
- **Path traversal**: `GET /../../etc/passwd` (y sus variantes codificadas, como `%2e%2e%2f`) nunca puede salir de `--root`. Canonicaliza la ruta y comprueba que sigue dentro de la raíz.
- `Content-Type` según la extensión (al menos html, css, js, json, png, jpg, svg, txt, pdf, wasm), `Content-Length` y `Date`.
- Ficheros grandes se envían por bloques, sin cargarlos enteros en memoria.
- Códigos mínimos: 200, 400, 403, 404, 405, 408, 414, 431, 500, 503, 505.

5. Conexiones y apagado

- *Keep-alive* por defecto en HTTP/1.1; `Connection: close` lo desactiva. En HTTP/1.0, cierre tras cada respuesta salvo `keep-alive` explícito.
- Timeout de lectura (configurable, 10 s por defecto): una conexión ociosa o un cliente que envía un byte por segundo (*slowloris*) no puede retener un worker para siempre.

- Ctrl+C: el servidor deja de aceptar conexiones, termina las peticiones en curso (máximo 5 s) y sale con código 0. Los hilos se unen (`join`) correctamente gracias a `Drop`.
- Log de acceso en stdout: `IP - "GET /ruta HTTP/1.1" 200 1532 0.4ms`.

6. Tests y rendimiento

- Tests unitarios del parser: al menos 40 casos (válidos, fragmentados, pipelined, malformados, límites).
- Tests de integración en `tests/` que arrancan el servidor en un puerto libre y le lanzan peticiones con `TcpStream`.
- `BENCH.md`: mide con `wrk` o `oha` (peticiones/s y latencia p99) variando el número de hilos, y explica dónde está el cuello de botella.

CAPÍTULO IV

Parte bonus

- **Fuzzing** del parser con `cargo-fuzz` (libFuzzer). Déjalo correr al menos una hora y documenta lo que encontró.
- Peticiones de rango (`Range: bytes=…`, respuesta `206`) y `If-Modified-Since` (`304`).
- Una segunda versión asíncrona con `tokio` en una rama aparte. Compara en `BENCH.md` las dos arquitecturas con 10 000 conexiones concurrentes.
- Si ya tienes un servidor propio en C o C++ (por ejemplo un clon de Redis), reescribe en Rust su parser de protocolo como biblioteca y compara la robustez de ambos fuzzeándolos.
- **Rust en el kernel**: si tu kernel de desarrollo tiene `CONFIG_RUST`, porta una versión simplificada de `ringdev` (sin `poll` ni `ioctl`) como módulo en Rust, partiendo de los ejemplos de `samples/rust/` de tu versión del kernel. Las abstracciones cambian rápido: documenta la versión exacta que usaste.

CAPÍTULO V

Evaluación

Prueba	Esperado
`curl -v http://localhost:8080/`	`200`, cabeceras correctas, contenido de `index.html`.
`curl --path-as-is http://localhost:8080/../../etc/passwd` y variantes codificadas	`403` o `404`. Nunca el fichero.
`printf 'GET / HTTP/1.1\r\nHost: x\r\n\r\nGET /a HTTP/1.1\r\nHost: x\r\n\r\n' | nc localhost 8080`	Dos respuestas en orden.
Petición enviada byte a byte con pausas de 1 ms	Se procesa correctamente.
Cliente que envía una cabecera cada 5 s	Cerrado por timeout; el worker queda libre.
`head -c 100000 /dev/urandom | nc localhost 8080`	`400` o cierre; el servidor sigue vivo.
Descarga de un fichero de 2 GiB con 4 clientes a la vez	Uso de memoria del servidor estable (decenas de MiB como mucho).
`wrk -t4 -c200 -d30s`	Sin errores de conexión; cifras coherentes con `BENCH.md`.

Prueba	Esperado
Ctrl+C durante una carga de `wrk`	Apagado ordenado en menos de 5 s, código 0.
`grep -rn unwrap src/`	Cada aparición justificada (solo donde un fallo es imposible por construcción).
Pregunta: ¿qué error de C de los proyectos anteriores habría evitado el compilador de Rust? ¿Cuál no?	Respuesta concreta con ejemplos propios.